Architecture 1001: x86-64 Assembly Calling Conventions

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

Calling Conventions

- 2 elements of importance:
  - *Register* conventions for which registers "belong" to the caller vs. callee
  - *Parameter-passing* conventions
- Both are compiler-dependent

Calling Conventions 2

- We're just talking about the MS "x64" ABI and System V "x86-64" ABI in the this section
- More examples at
  - http://en.wikipedia.org/wiki/X86_calling_conventions

"ABI" = Application Binary Interface. Standard of how executables are supposed to work on an architecture
https://raw.githubusercontent.com/wiki/hjl-tools/x86-psABI/x86-64-psABI-1.0.pdf
https://docs.microsoft.com/en-us/cpp/build/x64-software-conventions?view=msvc-160

Register Conventions: Caller-save registers

- Also called “volatile” registers by MS
  - I.e. the caller should assume they ***will*** be changed by the callee
- Registers "belong" to the callee.
- So the caller is in charge of saving the value before a call to a subroutine, and restoring the value after the call returns
- VisualStudio: RAX, RCX, RDX, R8, R9, R10, R11
- GCC: RAX, RDI, RSI, RDX, RCX, R8, R9, R10, R11

Register Conventions: Callee-save registers

- Also called “non-volatile” registers by MS
  - I.e. the caller should assume they ***will <u>not</u>*** be changed by the callee
- Registers "belong" to the caller
- If the callee needs to use more registers than are saved by the caller, the callee is responsible for making sure the values are stored/restored, so it doesn't break things for the caller
- VisualStudio: RBX, RBP, <u>RDI, RSI</u>, R12-R15
- GCC: RBX, RBP, R12-R15

Register Conventions: Balance

- Both caller and callee are responsible for *balancing* any register saves they perform (add to the stack), with restores (removal from the stack)
- Caller will *typically* save registers *right before the call* and restore *right after the call*
- Callee will *typically* save registers *at the beginning of the function* and restore *at the end of the function*

Register Conventions: Balance

- Both caller and callee are responsible for *balancing* any register saves they perform (add to the stack), with restores (removal from the stack)
- Caller will *typically* save registers *right before the call* and restore *right after the call*
- Callee will *typically* save registers *at the beginning of the function* and restore *at the end of the function*



VS vs. GCC et al.
Caller vs. Callee-save Regs

VS Caller-saved	RAX		RCX	RDX				R8-11	
VS Callee-saved		RBX			RSI	RDI	RBP		R12-15
System V (GCC et al.) Caller-saved	RAX		RCX	RDX	RSI	RDI		R8-11	
System V (GCC et al.) Callee-saved		RBX					RBP		R12-15

Parameter-passing Conventions: Common

- x86-64 compilers use a subset of the *caller-save* registers to pass parameters into and out of functions
- RAX or RDX:RAX passses out the return value
  - RAX holds anything 64 bits or smaller
  - RDX:RAX can be used to return 128 bit values

Parameter Conventions: Microsoft “x64” ABI

- First 4 parameters (from left to right) are put into RCX, RDX, R8, R9 respectively
- Remaining parameters “pushed” onto the stack so that the left-most parameter is at the lowest address
- Typically "mov" is used instead of "push" though

```
              RCX     RDX      R8       R9    "pushed"
int func(int a, int b, int c, int d, int e){
  int i = a+b-c+d-e;
  return i;
}
int main(){
  return func(0x11,0x22,0x33,0x44,0x55);
}
```

Parameter Conventions: System V “x86-64” ABI (GCC et al.)

- First 6 parameters (from left to right) are put into RDI, RSI, RDX, RCX, R8, R9 respectively
- Remaining parameters “pushed” onto the stack so that the left-most parameter is at the lowest address
- Typically "mov" is used instead of "push" though

```
              RDI     RSI     RDX     RCX      R8
int func(int a, int b, int c, int d, int e){
  int i = a+b-c+d-e;
  return i;
}
int main(){
  return func(0x11,0x22,0x33,0x44,0x55);
}
```

VS vs. GCC et al.
Function Parameter Order

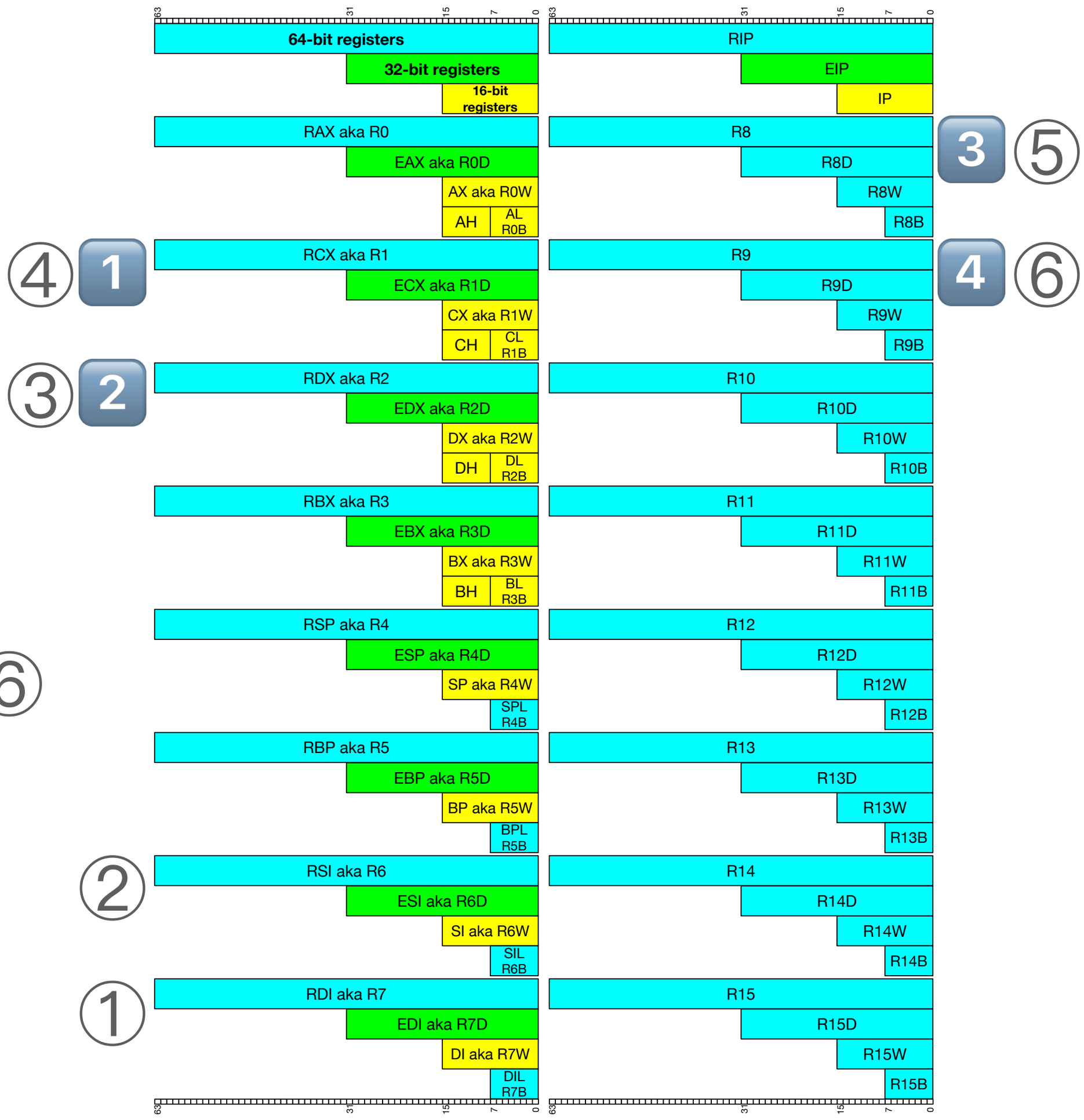
VS Caller-saved	RAX		RCX **(1)**	RDX **(2)**				R8 **(3)**	R9 **(4)**	R10-11	
VS Callee-saved		RBX			RSI	RDI	RBP				R12-15
System V (GCC et al.) Caller-saved	RAX		RCX **(4)**	RDX **(3)**	RSI **(2)**	RDI **(1)**		R8 **(5)**	R9 **(6)**	R10-11	
System V (GCC et al.) Callee-saved		RBX					RBP				R12-15

VS arguments

1 2 3 4

System V arguments

① ② ③ ④ ⑤ ⑥

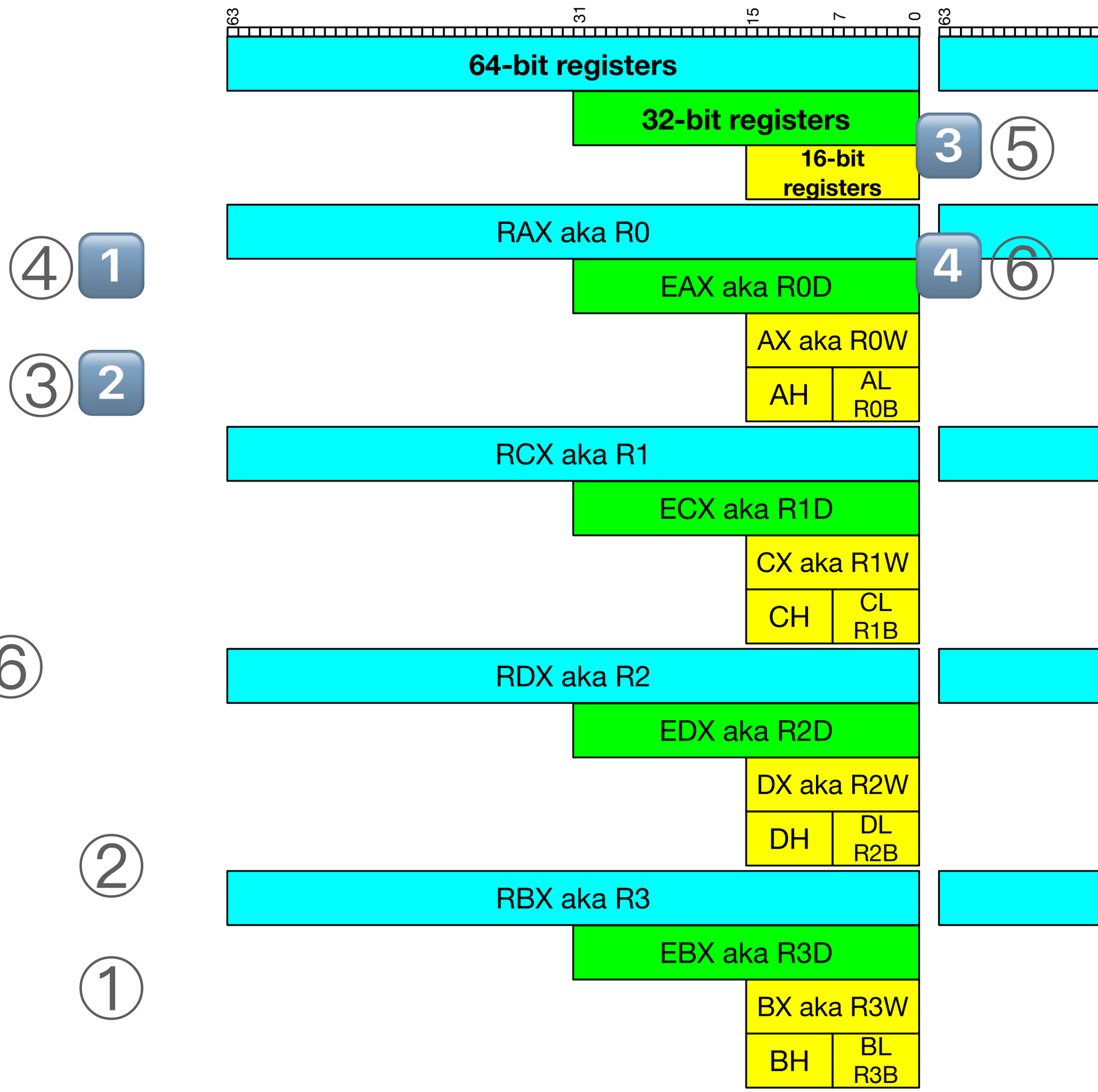


VS arguments

1 2 3 4

System V arguments

① ② ③ ④ ⑤ ⑥

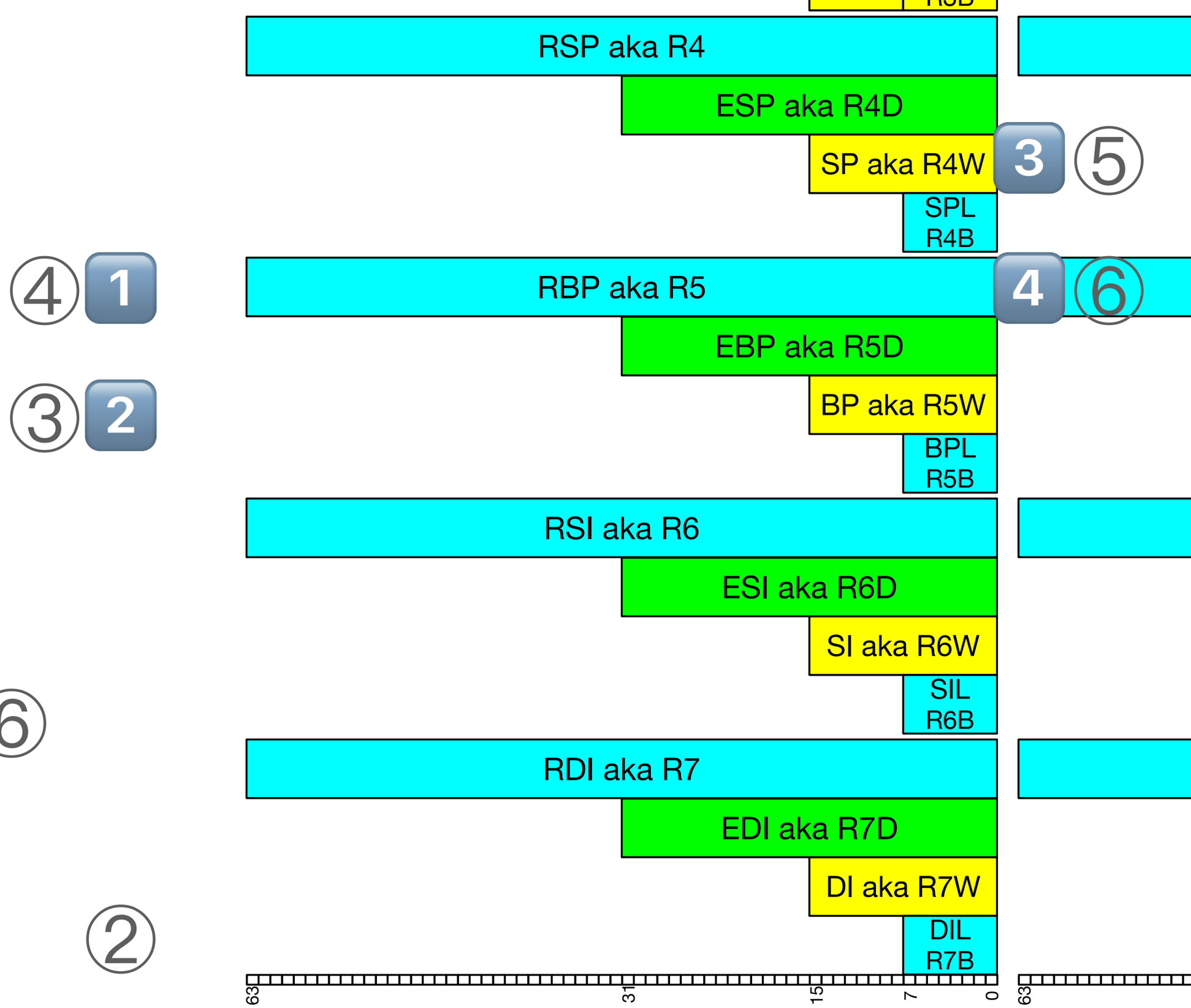


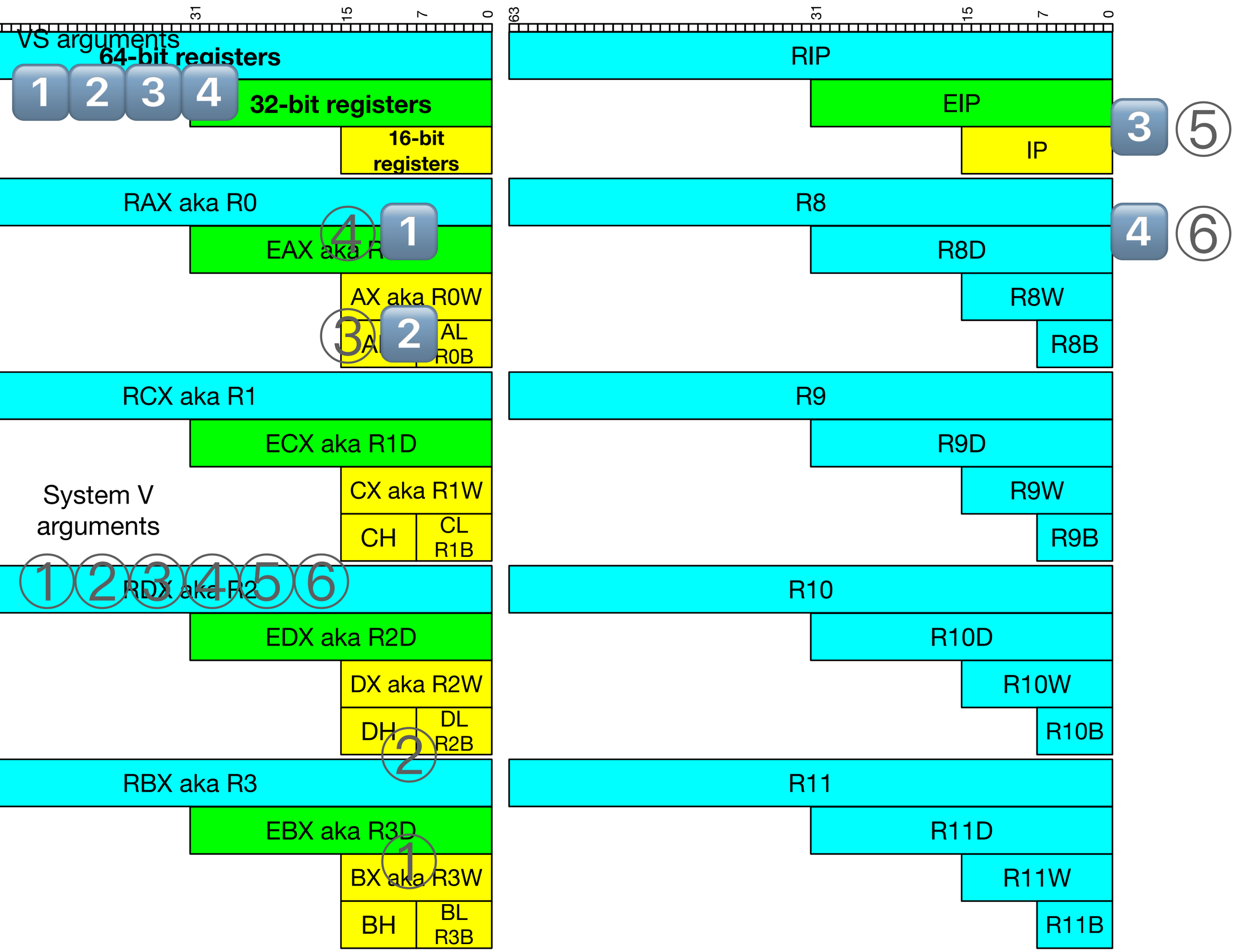
VS arguments

1 2 3 4

System V arguments

① ② ③ ④ ⑤ ⑥




31
15
7
0
63
31
15
7
0
VS arguments
64-bit registers
1 2 3 4
32-bit registers
16-bit registers
RIP
EIP
IP
3 5
RAX aka R0
EAX aka R
4 1
AX aka R0W
AH
AL R0B
3 2
R8
R8D
R8W
R8B
4 6
RCX aka R1
ECX aka R1D
CX aka R1W
CH
CL R1B
R9
R9D
R9W
R9B
System V arguments
1 2 3 4 5 6
RDX aka R2
EDX aka R2D
DX aka R2W
DH
DL R2B
2
R10
R10D
R10W
R10B
RBX aka R3
EBX aka R3D
BX aka R3W
BH
BL R3B
1
R11
R11D
R11W
R11B